Backend and Distributed Processing Specification

FastAPI Asynchronous Lifecycle, Celery Workers, Redis, and Idempotency

1. FastAPI Asynchronous Request Lifecycle

The backend utilizes Python 3.12, FastAPI, and asyncpg for non-blocking I/O. Endpoints are structured under /api/v1 with dependencies handling database session provisioning, JWT token authentication, and permission guards.

Connection pooling is managed via SQLAlchemy 2.0 AsyncEngine with pool_pre_ping enabled to detect dropped TCP connections proactively.

2. Transaction-Safe Task Dispatch

A critical architectural requirement is preventing race conditions between database transactions and background task dispatch. docvyn enforces transaction commitment before task dispatch:

```
db.add(doc)
await db.commit()      # Commit row to PostgreSQL first
await db.refresh(doc)
self.dispatch_processing_task(doc.id)  # Only dispatch after commit!
```

This guarantees that when the Celery worker queries Document by document_id, the record is guaranteed to exist.

3. Idempotency & Failure Recovery

The Celery ingestion task (process_document_task) is designed for strict idempotency: if re-delivered or manually retried, it purges any partial chunks from prior runs before inserting new ones. Celery uses task_acks_late=True, exponential retry backoff for transient connection errors, and immediate failure marking for permanent corrupt file errors.